



Template Injection / Unsafe Rendering Finding

Enterprise DevSecOps Security Lab

Finding ID: APP-03

Template Injection / Unsafe Rendering in Flask /hello Endpoint

HIGH RISK

Author / Assessor	Jagriti Banerjee
Assessment Area	Application Security / Flask Template Rendering
Affected Endpoint	/hello?name=<user-input>
Finding Type	Server-Side Template Injection / Unsafe Rendering
Severity	High
Primary CWE	CWE-1336 - Improper Neutralization of Special Elements Used in a Template Engine
OWASP Mapping	OWASP Top 10 A03:2021 Injection; ASVS input validation and output encoding controls

This document presents a professional finding write-up for the unsafe Flask rendering pattern identified in the vulnerable application. It explains the affected code path, proof-of-concept behavior, business impact, root cause, remediation strategy, DevSecOps controls, and retest criteria for closure.

Document Classification: Private Lab / Educational Security Assessment

1. Executive Summary

The vulnerable Flask application contains an unsafe template rendering issue in the /hello endpoint. User-controlled input is inserted into a template string and then passed to `render_template_string()`. Because the rendered string is interpreted by the Jinja template engine, attacker-supplied template syntax may be evaluated instead of being displayed as plain text.

In a safe proof-of-concept, the payload `{{7*7}}` would be evaluated and rendered as 49. This confirms that user input is not being treated strictly as data. Depending on the application context and available objects, server-side template injection can potentially lead to sensitive information disclosure, application logic manipulation, and in severe cases server-side code execution.

Field	Value
Finding ID	APP-03
Finding Name	Template Injection / Unsafe Rendering
Severity	High
CVSS-style Score	8.1 High
Affected File	src/app/app.py
Affected Route	/hello
Affected Function	hello()
Primary Risk	User input is interpreted as executable template syntax
Status	Open / Requires Remediation

8.1

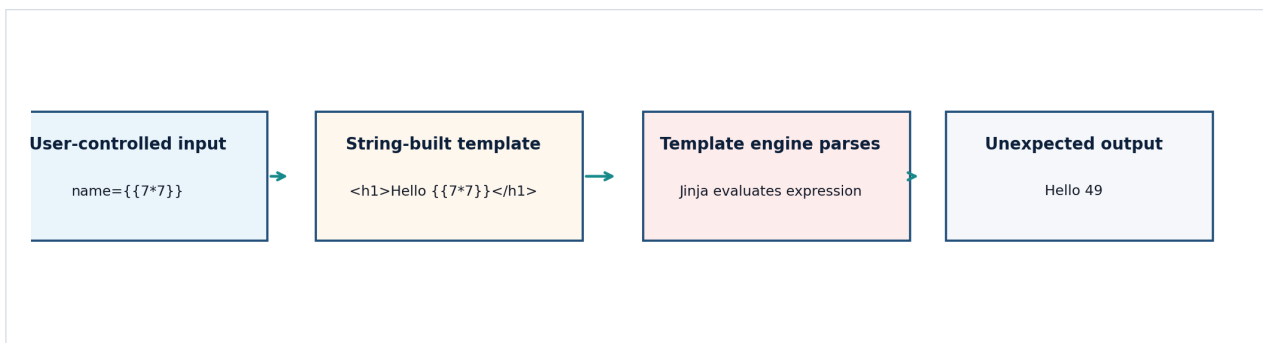


Figure 1 - High-level unsafe rendering attack flow.

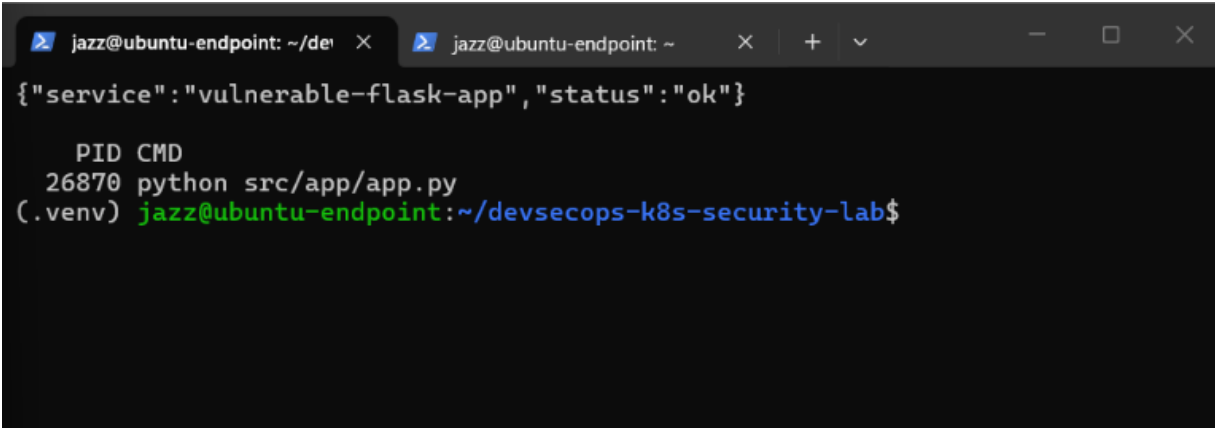
2. Affected Asset and Attack Surface

The affected component is a deliberately vulnerable Flask route created for the Enterprise DevSecOps lab. The endpoint accepts a name query parameter and returns a dynamically constructed HTML response.

Attribute	Details
Application	Enterprise DevSecOps Vulnerable Flask App
Technology Stack	Python, Flask, Jinja2 template engine, Docker, Kubernetes
Endpoint	GET /hello?name=<value>
Input Vector	name query parameter
Rendering Sink	render_template_string(template)
Exposure Type	Network-reachable HTTP endpoint inside the private lab

The vulnerable route is intentionally small, but it accurately models a common production issue: user input is mixed into a server-side template before rendering. In real environments, this pattern is dangerous because templates are not only presentation strings; they may also include expressions, filters, and access to application context.

Evidence Source

A terminal window screenshot showing a shell prompt. The first line displays a JSON status message: {"service": "vulnerable-flask-app", "status": "ok"}. The second line shows the command 'python src/app/app.py' being executed with PID 26870. The third line shows the shell prompt 'jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab\$' with a green prompt character and a blue path.

```
jazz@ubuntu-endpoint: ~/de  x  jazz@ubuntu-endpoint: ~  x  +  v  -  □  x
{"service": "vulnerable-flask-app", "status": "ok"}
PID CMD
26870 python src/app/app.py
(.venv) jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
```

Figure 2 - Vulnerable Flask application running in the private lab environment.

3. Technical Finding Details

The vulnerable implementation constructs a template with an f-string and embeds untrusted user input directly inside that template. The resulting string is then passed to the Jinja rendering engine.

```
@app.route("/hello")
def hello():
    name = request.args.get("name", "guest")

    template = f"<h1>Hello {name}</h1>"
    return render_template_string(template)
```

This is unsafe because the application does not render a static template with a variable. Instead, it builds the template itself using untrusted input. When the template engine processes the final string, any template expression supplied by the user may be interpreted as template syntax.

Root Cause	Explanation
String-built template	The template body is assembled dynamically with user input instead of using a static template and a safely passed variable.
Unsafe rendering sink	render_template_string() evaluates the final template string. If the string contains attacker-controlled template syntax, it may execute in the template context.
Missing output handling	The application does not force user input to be treated only as text before rendering.
No negative test coverage	The test plan did not initially include a test to ensure template syntax is displayed literally rather than evaluated.

Safe Proof-of-Concept

The following non-destructive payload is sufficient to demonstrate template evaluation without reading files, executing system commands, or accessing sensitive data.

```
Request:
curl "http://localhost:5000/hello?name={{7*7}}"
```

```
Vulnerable behavior:
<h1>Hello 49</h1>
```

```
Expected secure behavior:
<h1>Hello {{7*7}}</h1>
```

Template Injection Evidence - Unsafe render_template_string Sink

Source code sink (src/app/app.py)

```
@app.route("/hello")
def hello():
    name = request.args.get("name", "guest")

    template = f"<h1>Hello {name}</h1>"
    return render_template_string(template)
```

Safe PoC payload (non-destructive):

```
$ curl "http://localhost:5000/hello?name={{7*7}}"
```

```
<h1>Hello 49</h1>
```

Interpretation: user input was parsed by Jinja as template syntax.
Expected secure behavior: the response should display literal {{7*7}}, not 49.

Figure 3 - Source-code sink and safe PoC behavior for template expression evaluation.

4. Security Impact Analysis

Template injection is high impact because it converts a harmless-looking display endpoint into a server-side evaluation point. The practical impact depends on the available template context, framework configuration, exposed objects, and runtime restrictions. Even when full command execution is not proven, the issue should be treated seriously because the application has crossed a critical trust boundary: user input is being interpreted as server-side logic.

Impact Area	Potential Consequence
Confidentiality	Exposure of environment values, application configuration, debug data, object properties, or internal variables if accessible through the template context.
Integrity	Manipulation of rendered output, abuse of template logic, or unexpected behavior in pages that trust rendered server-side content.
Availability	Template expressions or loops may cause rendering errors, excessive processing, or denial-of-service conditions in weakly controlled contexts.
Exploit Chain Risk	May be combined with debug exposure, unsafe objects, command injection, weak container isolation, or leaked secrets.
Compliance / SDLC Risk	Fails secure coding expectations for output encoding, template safety, and trust-boundary handling.

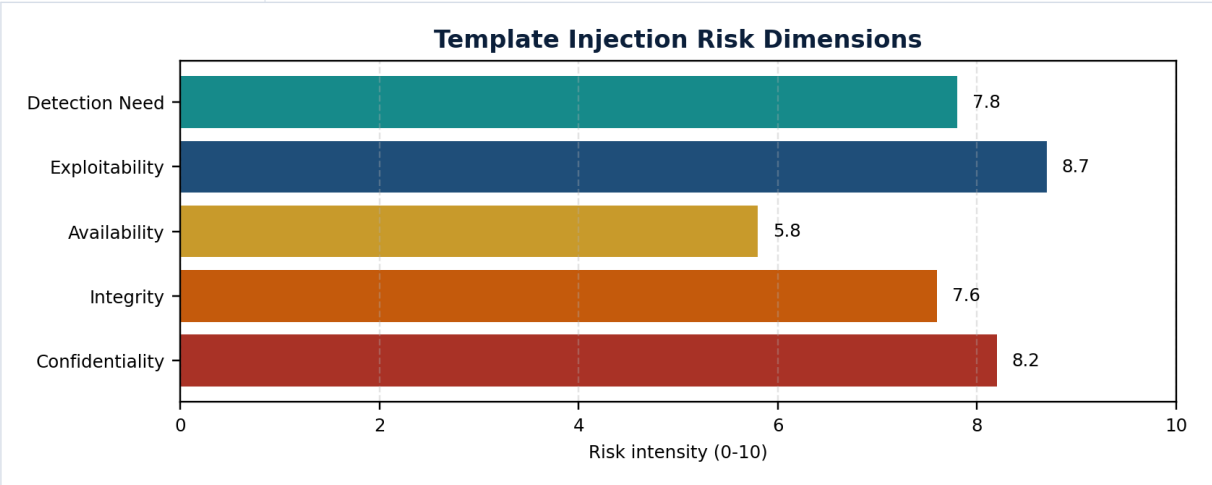


Figure 4 - Risk intensity by security impact dimension.

Business Risk Statement

If this pattern were present in a production-facing application, attackers could use template syntax to probe the server-side rendering context. Successful exploitation may reveal sensitive configuration, enable application logic abuse, or become a stepping stone toward deeper compromise. The finding therefore requires priority remediation before production deployment.

5. OWASP ASVS, OWASP Top 10, and CWE Mapping

The finding maps to secure development requirements related to input validation, output encoding, injection prevention, and safe server-side rendering. The mapping below is written for portfolio and assessment documentation, and should be aligned with the exact ASVS version adopted by the organization during formal audits.

Standard	Mapping	Rationale
CWE	CWE-1336	Improper Neutralization of Special Elements Used in a Template Engine. The user input is interpreted by a template engine.
CWE	CWE-94 / CWE-95 family	Relevant when template evaluation permits code generation or expression evaluation beyond intended display behavior.
OWASP Top 10	A03:2021 - Injection	Template injection is a form of injection where untrusted data is interpreted by a server-side interpreter.
OWASP ASVS	Input Validation / Sanitization controls	User input must be validated and constrained before being used in sensitive rendering contexts.
OWASP ASVS	Output Encoding / Injection Prevention controls	User-supplied output should be contextually encoded and passed as data, not merged into executable template syntax.
OWASP ASVS	Secure Development / Error Handling controls	Unexpected template parsing behavior must be avoided and covered by secure tests.

CVSS-Style Scoring Rationale

Metric	Assessment
Attack Vector	Network - exploitable through HTTP request to /hello
Attack Complexity	Low - payload can be supplied directly in query parameter
Privileges Required	None - no authentication exists in the lab route
User Interaction	None - server renders request directly
Confidentiality	High potential depending on template context
Integrity	High potential if server-side expressions influence output or logic
Availability	Low to Medium potential through rendering errors or expensive expressions
Overall Score	8.1 High, conservative lab scoring

6. Recommended Remediation

The safest remediation is to stop building templates from user-controlled input. The application should render a static template and pass the user value as a variable. The template engine can then apply escaping rules to the variable instead of interpreting the variable content as template syntax.

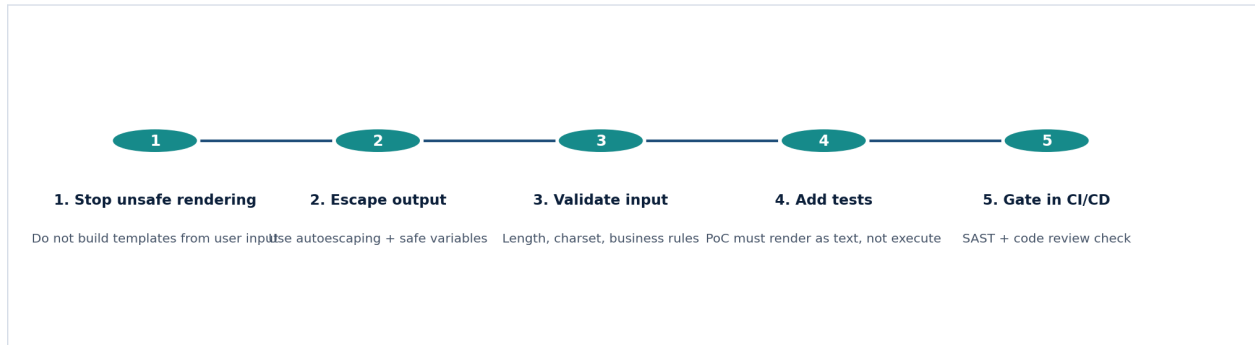


Figure 5 - Recommended remediation sequence.

Unsafe Pattern

```
# Do not do this
name = request.args.get("name", "guest")
template = f"<h1>Hello {name}</h1>"
return render_template_string(template)
```

Preferred Secure Pattern

```
# Safer pattern: static template, user input passed as data
from flask import render_template

@app.route("/hello")
def hello():
    name = request.args.get("name", "guest")
    return render_template("hello.html", name=name)

# templates/hello.html
# <h1>Hello {{ name }}</h1>
```

Alternative Minimal Fix for Lab Demonstration

If the lab intentionally avoids file-based templates, the template should remain static and user data should be passed as a variable, not inserted into the template body before rendering.

```
# Safer render_template_string pattern for small lab demo
@app.route("/hello")
def hello():
    name = request.args.get("name", "guest")
    return render_template_string("<h1>Hello {{ name }}</h1>", name=name)
```

Additional Hardening Controls

- Limit input length for the name parameter and reject unexpected characters where business logic allows.
- Keep Flask debug mode disabled outside controlled local testing.
- Avoid exposing sensitive objects, configuration, or helper functions to templates unless strictly required.
- Add unit tests that verify template syntax in user input is displayed literally and not evaluated.
- Add code review rules to flag `render_template_string()` usage involving user-controlled data.
- Use SAST rules and CI/CD checks to fail builds when unsafe rendering sinks are introduced.

7. Retest Plan and Closure Criteria

The finding should only be closed after functional retesting confirms that user-controlled template syntax is no longer evaluated by the server. The test should be automated and included in the CI/CD pipeline so the issue does not reappear during future changes.

Test Case	Request / Action	Expected Result
TC-SSTI-001	GET /hello?name={{7*7}}	Response displays literal {{7*7}} or an encoded equivalent. It must not return 49.
TC-SSTI-002	GET /hello?name={{config}}	Response must not disclose configuration or server-side objects.
TC-SSTI-003	GET /hello?name=<script>alert(1)</script>	HTML/JavaScript is safely encoded and not executed by the browser.
TC-SSTI-004	Review app.py for render_template_string with f-string/user input	No dynamic template construction from request-controlled input remains.
TC-SSTI-005	Run CI/CD SAST and unit tests	Pipeline passes only when unsafe rendering pattern is absent and negative tests pass.

Closure Requirements

- Code changed to render a static template and pass user data as a variable.
- Safe proof-of-concept payload no longer evaluates server-side expressions.
- Unit tests and AppSec test cases added for unsafe rendering regression coverage.
- Security report evidence updated with before-and-after screenshots or terminal output.
- CI/CD pipeline stores SAST and test artifacts for auditability.

8. DevSecOps Pipeline Control Recommendations

Because this project is an Enterprise DevSecOps lab, remediation should not stop at a one-time code fix. The corrected control should be enforced through secure SDLC gates so the same class of issue is detected early in future commits.

Control Area	Recommended Implementation
Secure coding standard	Document a rule that user input must never be concatenated into server-side templates.
Pull request review	Require reviewers to inspect rendering functions, template helpers, and any <code>render_template_string()</code> usage.
SAST	Add Semgrep/Bandit-style rules that flag dynamic template rendering using request parameters.
Unit tests	Add negative test payloads such as <code>{{7*7}}</code> and verify literal output.
DAST smoke test	Run curl-based tests against the <code>/hello</code> endpoint in the private lab environment.
Container hardening	Continue running the app as non-root with read-only filesystem and minimal capabilities.
Runtime detection	Monitor for suspicious template errors or abnormal endpoint behavior in logs and dashboards.



```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ bandit -r src/app/
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b682_subprocess_
popen_with_shell_equals_true.html
Location: src/app/app.py:55:13
54     f'ping -c 1 {host}"',
55     shell=True,
56     capture_output=True,
57     text=True
58 )
59
60     return f"<pre>[result.stdout]\n[result.stderr]</pre>"
61

>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interf
aces.
Severity: Medium Confidence: Medium
CWE: CWE-605 (https://cwe.mitre.org/data/definitions/605.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/B104_hardcoded_b
ind_all_interfaces.html
Location: src/app/app.py:73:17
72     debug_mode = os.getenv("FLASK_DEBUG", "false").lower() == "true"
73     app.run(host="0.0.0.0", port=5000, debug=debug_mode)

Code scanned:
Total lines of code: 70
Total lines skipped (#nosec): 0

Run metrics:
Total issues (by severity):
  Undefined: 0
  Low: 1
  Medium: 2
  High: 1
Total issues (by confidence):
  Undefined: 0
  Low: 1
  Medium: 1
  High: 2
Files skipped (0):
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 6 - SAST evidence from the DevSecOps lab pipeline. Additional rules should be added for unsafe template rendering patterns.

9. Evidence Summary and Appendix

The evidence set for this finding combines source-code review, safe proof-of-concept behavior, application runtime validation, and SAST pipeline context. Evidence should be stored with the project repository under evidence/screenshots and security/reports for traceability.

Evidence Item	Description
Source-code sink	src/app/app.py shows request input inserted into a dynamically built template and rendered with render_template_string().
Safe PoC	Payload {{7*7}} demonstrates server-side expression evaluation without destructive actions.
Application running	Private lab screenshot confirms the vulnerable Flask application was running locally.
SAST context	Bandit/SAST screenshots demonstrate that static analysis was part of the project workflow.
Remediation criteria	Retest table defines closure expectations for engineering and security validation.

Recommended Secure Code Diff

```
- template = f"<h1>Hello {name}</h1>"
- return render_template_string(template)
+ return render_template_string("<h1>Hello {{ name }}</h1>", name=name)
```

Final Risk Position

The issue is rated High until the rendering implementation is changed and retested. After remediation, the residual risk can be reduced to Low if user-controlled template syntax is rendered as text, SAST rules are in place, and negative regression tests are enforced in CI/CD.

End of Template Injection / Unsafe Rendering Finding Report